

Lab Work Assignment 2: Automating Dataset Extension

Course: ML Operations

Objective:

The goal of this lab is to design, implement, and evaluate a system for automating the extension of datasets. Students are expected to create a configuration-driven approach for selecting data batches, combine these batches into dynamic training and validation sets while keeping the test set static, and demonstrate how different subsets of data affect model performance.

Resources:

- Example of dataset extension script and configuration management: [Sample Notebook](#)
- Training loop with PyTorch: [PyTorch Tutorial](#)
- MNIST dataset: [Kaggle MNIST Dataset](#)
- Rice types dataset: [Kaggle Rice Types Dataset](#)
- CIFAR dataset: [CIFAR Dataset](#)

Submission Deadline:

Submit your completed project and lab report by 23.09.2024.

Part 1: Dataset Configuration and Management

Data Batch Configuration:

- **Task:** Create a configuration file (e.g., `config.yaml` or `config.json`) that allows the selection of data batches from a larger dataset. This configuration should include parameters for the number of batches to be used for training, validation, and testing.

Data Combination Script:

- **Task:** Implement a script that reads the configuration file and combines the selected batches into:
 - Dynamic training and validation sets
 - A static test set

Script Requirements:

- The test set should remain static to provide a consistent evaluation metric across different training configurations.
 - The training and validation sets should be created dynamically based on the configuration file.
-

Part 2: Model Training and Evaluation

Training Procedure:

- **Task:** Modify the training procedure to accommodate the dynamically generated training and validation sets. Ensure that your model can be trained on these datasets and evaluated properly.

Performance Evaluation:

- **Task:** Demonstrate how different subsets of data batches affect model performance. Train the model with various configurations of training and validation sets and evaluate the performance on the static test set.

Metrics to Report:

- Include performance metrics such as accuracy, precision, recall, and F1-score. Analyze how changes in the size and composition of training batches impact these metrics.
-

Part 3: Code Management and Best Practices

Configuration Management:

- **Task:** Ensure that all data batch selection and combination parameters are controlled via configuration files, making the process flexible and reproducible.

Logging:

- **Task:** Use Python's logging module to track key events, such as the start and end of data processing, training, and evaluation.

Code Quality:

- **Task:** Implement code quality checks using linters and formatters like mypy, ruff, black, and isort. Adhere to best practices in code style and structure.

Dependency Management:

- **Task:** Use poetry for managing project dependencies to ensure a reproducible environment.

Version Control:

- **Task:** Use GitHub (or a similar platform) to manage your code. Commit your code regularly and provide meaningful commit messages.
-

Part 4: Lab Report

Content:

1. **Introduction:**
 - Describe the problem domain, the dataset used, and the goals of your dataset extension approach.
2. **Pipeline Description:**
 - Outline the structure of your dataset extension pipeline, including the configuration management, data combination script, and how the training and validation sets are created.
3. **Model Evaluation:**
 - Present and discuss the results of your model, including metrics and performance insights. Highlight how different batch configurations affected the performance.
4. **Best Practices:**
 - Explain how you implemented best practices in configuration management, logging, code quality, and version control.
5. **Reflection:**
 - Reflect on the effectiveness of your dataset extension approach and suggest any improvements that could be made with additional resources or time.

Code Submission:

- Submit your entire project repository, including all code, configuration files, and any generated datasets or artifacts.
- Ensure your GitHub repository is well-organized with a clear commit history and documentation.

Grading Criteria:

1. **Code Quality and Management:** Proper use of configuration files, logging, code linters, formatters, and version control.
2. **Dataset Extension Implementation:** Effective implementation of dataset extension, including the dynamic creation of training and validation sets and a static test set.
3. **Model Training and Evaluation:** Quality of the model's performance metrics and the analysis of how different data batches affect performance.
4. **Report Quality:** Clarity and thoroughness of the lab report, including the reflection on the dataset extension approach.